

Reporte de Práctica de Laboratorio — Programación Orientada a Objetos

EE: Temas Selectos de IIE II (Código: IEDI 18005)

Facultad de Instrumentación Electrónica — Universidad Veracruzana

Información General del Equipo

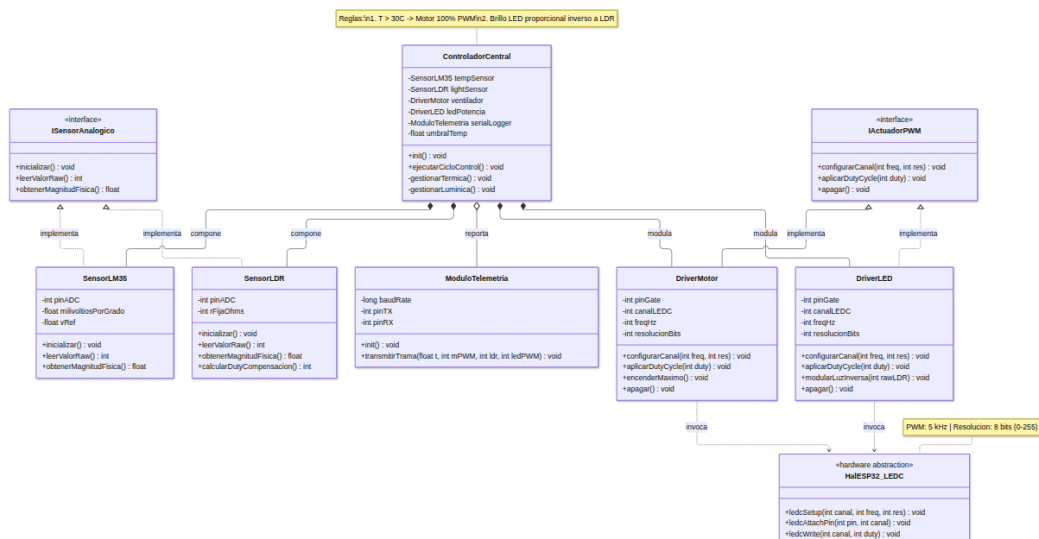
- **Número de Práctica:** Práctica 1
- **Título de la Práctica:** Invernadero con ESP32
- **Fecha de Entrega:** [28/09/2026]
- **Enlace al Repositorio de GitHub:** https://github.com/DarkZodiac0/mirepositorio_poo

Nombre Completo del Integrante	Matrícula UV	Usuario de GitHub
Pérez Montero Axel Arturo	[S23013994]	@DarkZodiac0
Bremont Limon Isaac Gabriel	[S23013924]	@Isaacclimon14

Sección 1: Mapeo de Abstracción (Hardware → POO)

En este apartado demostrarán cómo convirtieron sus componentes físicos (sensores, actuadores, controladores) en objetos lógicos dentro de su programa de software en C++ o Python

Arquitectura y Diseño Orientado a Objetos del Sistema



Sección 2: El Registro de Errores ("Bug Log")

La depuración e integración física es el núcleo de la instrumentación electrónica. Reporten al menos dos fallas o comportamientos inesperados (lógicos o de hardware) que sufrieron durante la integración del circuito con su código y cómo los solucionaron reprogramando o ajustando su POO.

#	Falla Física o Lógica Detectada	Causa Raíz (Hardware / Software)	Solución Técnica Implementada en Código / Conexión
1	<i>Ejemplo: El sensor de humedad devolvía constantes de 0% de humedad de forma fija.</i>	<i>La instancia del objeto no inicializaba el convertidor analógico-digital (ADC) debido a que el constructor de la clase se ejecutaba antes de que se estableciera la energía del periférico.</i>	<i>Se separó la configuración del hardware del constructor en un método específico llamado <code>sensor.begin()</code>, el cual es llamado de manera explícita dentro del setup general del programa.</i>
2	<i>Ejemplo: Que tuvimos que ajustar la sensibilidad de la fotoresistencia varias veces</i>	<i>Una error de calculo para determinar la sensibilidad y el valor de la resistencia</i>	<i>Tuvimos que ajustar la sensibilidad en el código y también cambiamos el valor de la resistencia</i>

Sección 3: Justificación del Diseño Orientado a Objetos

1. Herencia y Reutilización de Código:

- **Respuesta:** No se utilizó herencia ni Programación Orientada a Objetos (POO). El código sigue un paradigma de **programación imperativa y modular plana** (estilo estándar de Arduino/ESP32 con `setup()` y `loop()`). Este enfoque fue preferible debido a que el sistema es de control directo en tiempo real, con una cantidad reducida y fija de periféricos (un motor, un LED, un LDR y un encoder). Implementar jerarquías de clases y métodos virtuales habría agregado sobrecarga de memoria innecesaria (tablas vtable) y mayor latencia sin aportar ventajas reales de escalabilidad para la escala actual del proyecto.
-

2. Polimorfismo:

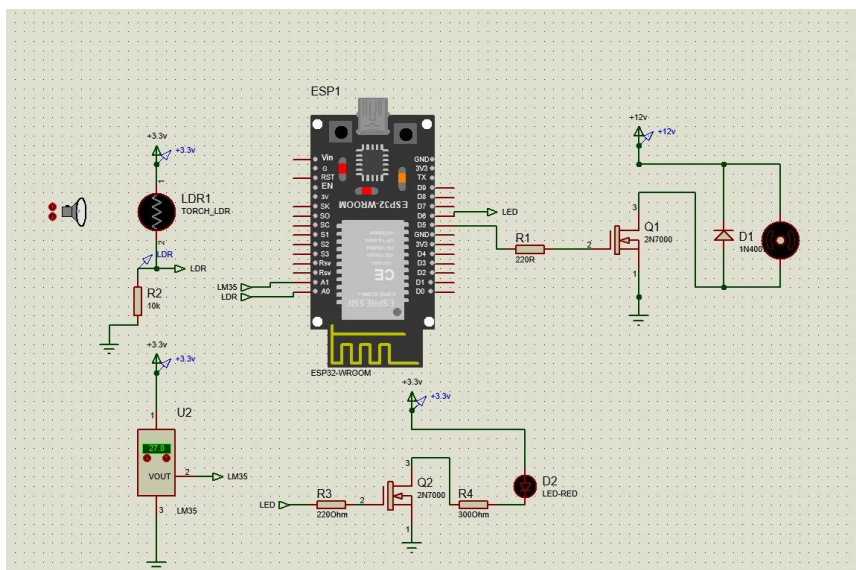
- **Respuesta:** No se implementó polimorfismo dinámico basado en clases, ya que no existen objetos definidos en el código. El único comportamiento de "múltiples formas" presente se da a nivel funcional mediante la API del Core de ESP32 (`ledcWrite`), la cual aplica la misma señal de ciclo de trabajo a dos periféricos físicamente distintos (`PIN_MOTOR_PWM` para controlar la velocidad del motor mediante el driver y `PIN_LED` para la conmutación rápida del MOSFET IRLZ44N). No obstante, esto corresponde a una reutilización de funciones del framework y no a polimorfismo de POO.
-

3. Encapsulamiento y Ocultamiento de Datos:

- **Respuesta:** No se utilizaron modificadores de acceso (`private` o `protected`) ni métodos *Getters/Setters*, ya que no se estructuraron clases. Sin embargo, la integridad del hardware y la seguridad de las variables críticas se gestionan mediante mecanismos a nivel de lenguaje procedural:
- **Inmutabilidad de pines y parámetros:** Se utilizó el calificador `const` para pines y configuración PWM (`PIN_MOTOR_PWM`, `PIN_LED`, `FREC_PWM`, etc.), impidiendo su reasignación o corrupción en tiempo de ejecución.
- **Integridad en interrupciones:** La variable `contadorPulsos` está calificada como `volatile` para garantizar lecturas consistentes frente a la ISR.
- **Protección física del actuador:** Se implementó `constrain(valorLDR, minLDR, maxLDR)` como barrera de software para acotar las lecturas analógicas dentro de rangos operativos seguros antes del mapeo PWM, evitando sobrecargas o ciclos de trabajo fuera de especificación.

Sección 4: Evidencias de Funcionamiento (Validación Dual)

Esta sección contiene los enlaces y capturas obligatorios que validan la ejecución real del software interactuando con el hardware.



4.1. Captura de Pantalla de la Terminal / Monitor Serial

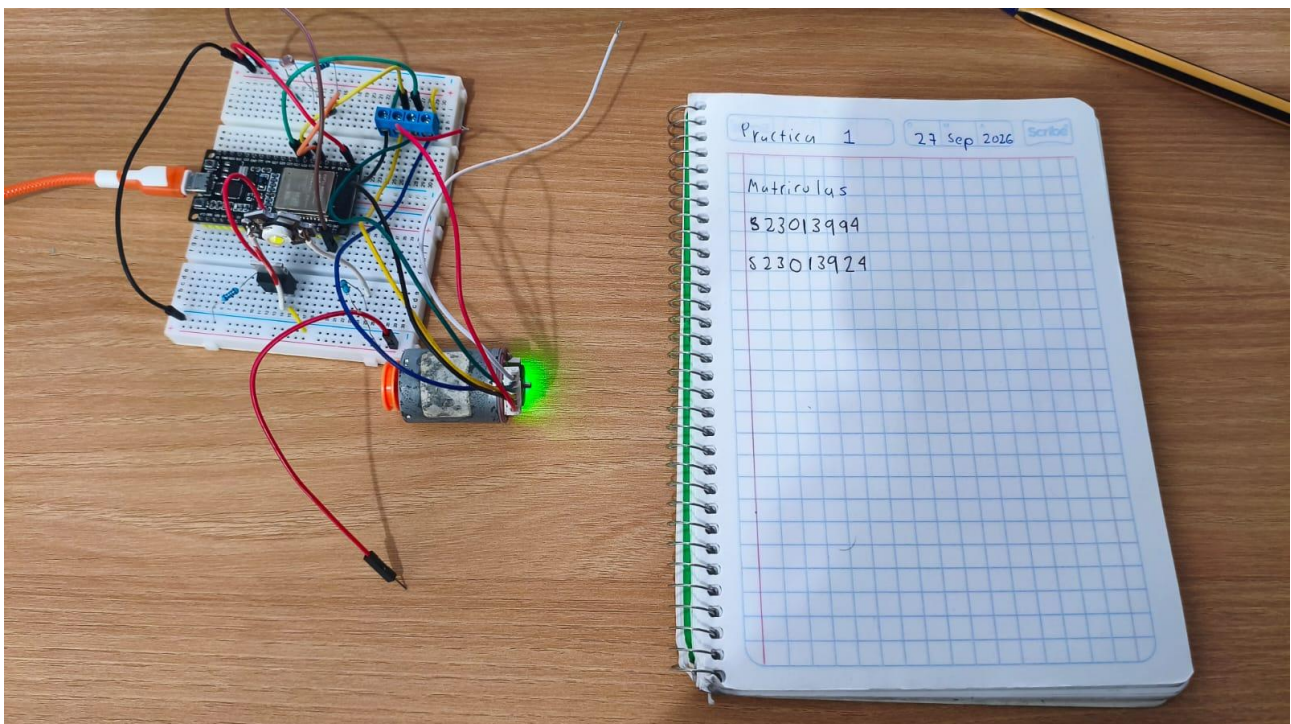
```
axstar@axstar-ASUS-TUF-Gaming-F16-  
=====   
SIMULADOR DE HARDWARE INDUSTRIAL (SISTEMA HMI)   
=====   
Modo de Operación : [MANUAL]   
Sensor Temperatura: 25.00 °C [Rango: 0.0 - 150.0 °C]   
Sensor Presión : 1.00 Bar [Rango: 0.0 - 15.0 Bar]   
Bomba Enfriamiento: 0.0 % [Modulación Proporcional PWM]   
Válvula de Alivio : CERRADA (0) [Control Digital ON/OFF]   
-----   
>> Sistema inicializado. Listo para recibir instrucciones.   
-----   
Comandos HMI disponibles:   
- Lectura : 'leer caudal' | 'leer manometro'   
- Modos : 'modo manual' | 'modo auto' | 'modo pruebas'   
- Actuadores (M) : 'set bomba <0-100>' | 'set valvula <0/1>'   
- Variables (M) : 'set temp <0-150>' | 'set pres <0-15>'   
- Salir : 'salir'   
=====   
HMI Consola >> modo manual
```

```
axstar@axstar-ASUS-TUF-Gaming-F16-  
=====   
SIMULADOR DE HARDWARE INDUSTRIAL (SISTEMA HMI)   
=====   
Modo de Operación : [AUTOMATICO]   
Sensor Temperatura: 22.50 °C [Rango: 0.0 - 150.0 °C]   
Sensor Presión : 1.50 Bar [Rango: 0.0 - 15.0 Bar]   
Bomba Enfriamiento: 40.0 % [Modulación Proporcional PWM]   
Válvula de Alivio : CERRADA (0) [Control Digital ON/OFF]   
-----   
>> [MANÓMETRO]: 1.57 Bar   
-----   
Comandos HMI disponibles:   
- Lectura : 'leer caudal' | 'leer manometro'   
- Modos : 'modo manual' | 'modo auto' | 'modo pruebas'   
- Actuadores (M) : 'set bomba <0-100>' | 'set valvula <0/1>'   
- Variables (M) : 'set temp <0-150>' | 'set pres <0-15>'   
- Salir : 'salir'   
=====   
HMI Consola >>
```

```
axstar@axstar-ASUS-TUF-Gaming-F16-

=====
SIMULADOR DE HARDWARE INDUSTRIAL (SISTEMA HMI)
=====
Modo de Operación : [PRUEBAS]
Sensor Temperatura: 88.50 °C [Rango: 0.0 - 150.0 °C]
Sensor Presión : 12.30 Bar [Rango: 0.0 - 15.0 Bar]
Bomba Enfriamiento: 100.0 % [Modulación Proporcional PWM]
Válvula de Alivio : ABIERTA (1) [Control Digital ON/OFF]
=====
[!WARNING!] INTERLOCK DE SEGURIDAD ACTIVADO (PRIORIDAD ALTA)
Condición crítica detectada: Temp > 85.0 °C o Presión > 12.0 Bar
Acción automática : Bomba al 100% | Válvula ABIERTA
=====
>> MODO DE PRUEBAS: Fallo inyectado (Temp = 92.0 °C, Presión = 13.5 Bar).
=====
Comandos HMI disponibles:
- Lectura : 'leer caudal' | 'leer manometro'
- Modos : 'modo manual' | 'modo auto' | 'modo pruebas'
- Actuadores (M) : 'set bomba <0-100>' | 'set valvula <0/1>'
- Variables (M) : 'set temp <0-150>' | 'set pres <0-15>'
- Salir : 'salir'
=====
HMI Consola >> 
```

4.2. Fotografía del Circuito Físico Armado (Garantía de Autoría)



4.3. Enlace Directo al Video Demostrativo (Máximo 1.5 minutos)

*Coloca el enlace directo de tu video demostrativo subido a tu nube institucional (Drive/OneDrive) o YouTube. Asegúrate de configurar los permisos en modo **Público**.*

- **Enlace de Visualización:**

https://uvmx-my.sharepoint.com/:f/g/personal/zs23013994_estudiantes_uv_mx/IgALcuGM6ao_QJonw9Hrh7pOAYMEKEU4WteC_2GGREEaQM?e=8kfBsN

https://uvmx-my.sharepoint.com/:f/g/personal/zs23013923_estudiantes_uv_mx/IgCJSs6NZcKaQ5kKsbsS0mNpAY4oFaGnhEbxaqQILeCIXX4?e=tjRf6n